



SUBJECT :- DATA WAREHOUSE AND MINING

ASSIGNMENT NUMBER:- 10

AIM:- To develop an integrated web-based system that applies major data warehousing and data mining techniques to discover cyberattack patterns across networks

NAME :- Abhinav Mishra

CLASS :- CE-1

ROLL NO :- 51

BATCH :- B

PRN :- 2414110249

1. Project Overview

Aim

To develop an integrated web-based system that applies the major data warehousing and data mining techniques (preprocessing, a star-schema data warehouse with OLAP, association rule mining, J48 and Naive Bayes classification, regression and clustering) to network-intrusion and cyber-incident data, and presents the results through an interactive dashboard.

Abstract

Every network that is connected to the internet is probed, flooded and attacked, and each attack leaves a trace in connection logs and incident reports. Mining these records shows which attacks hit which network, what traffic features give them away, when they happen and how much they cost. This mini project, **Cyberattack Pattern Mining Across Networks**, brings the data mining methods studied in the previous practical assignments together in one application.

The system is built with Python and Streamlit and works on two bundled datasets: DS1, 30,180 network connection records captured on five networks (Corporate LAN, University campus, Cloud data-centre, IoT smart-grid and Banking core) and labelled Normal, DoS, Probe, R2L, U2R or Botnet; and DS2, 5,000 organisation-level security incidents from 2019 to 2025 with sector, attack vector, threat actor, impact and financial loss. Users can also upload their own CSV or Excel file and run every technique on it. Interactive Plotly charts show the results, and a comparison page evaluates the classifiers side by side to identify the best-suited model.

Problem Statement

When each data mining algorithm is studied in a separate assignment on a separate dataset, it is hard to see how the techniques complement one another and which one suits a given question. Security data makes this harder, because the raw logs are noisy, heavily skewed and different on every network. This project addresses the problem by applying all of the techniques to a common pair of cyber-security datasets in a single application, where the patterns found by one method can be checked against the others.

Objectives

- Create two structured cyber-security datasets (network traffic and security incidents) and allow user uploads.
- Inspect and preprocess the data: missing values, duplicates, outliers, normalisation, discretisation, feature selection and PCA.
- Design a star-schema data warehouse and demonstrate the OLAP operations roll-up, drill-down, slice, dice and pivot.
- Discover attack signatures with Apriori and FP-Growth, and compare the rules across networks.
- Classify connections and incidents with a J48 (C4.5) decision tree and a Naive Bayes classifier.
- Predict traffic volume and incident financial loss with regression models.
- Group connections and incidents with K-Means, hierarchical clustering and DBSCAN.
- Compare models within each task and recommend the best-suited model from measured results.

2. Dataset and Project Scope

The application uses two CSV datasets stored in the data/ folder. Both are synthetic: they are produced by data/generate_data.py with a fixed random seed, so every run gives the same data. DS1 follows the NSL-KDD / KDD Cup '99 connection schema. A small number of missing values and duplicate rows are added on purpose so that the preprocessing steps have something to fix. Each technique selects the columns it needs from these datasets.

DS1 – Network Intrusion Traffic (network_traffic.csv)

Table 1. Attributes of DS1 (23 columns).

Attribute	Type	Description
conn_id	Identifier	Unique connection identifier.
timestamp	Date-time	Connection start time (UTC, August 2026).
network	Categorical	Monitored network: Corporate-LAN, University-Campus, Cloud-DC, IoT-Grid, Banking-Core.
src_country	Categorical	Geo-located source of the connection.
protocol	Categorical	Transport protocol (tcp, udp, icmp).
service	Categorical	Destination service (http, ssh, dns, ftp, telnet, mqtt ...).
flag	Categorical	TCP connection status flag (SF, S0, REJ, RSTO ...).
duration	Numeric	Connection length in seconds.
src_bytes / dst_bytes	Numeric	Bytes sent source → destination and destination → source.
packets	Numeric	Total packets exchanged (regression target).
count / srv_count	Numeric	Connections to the same host / same service in the past 2 seconds.
serror_rate / rerror_rate	Numeric	Share of connections with SYN errors / REJ errors.
same_srv_rate	Numeric	Share of connections to the same service.
dst_host_count	Numeric	Distinct connections to the destination host.
failed_logins	Numeric	Failed login attempts.
logged_in / root_shell	Binary	1 if login succeeded / 1 if a root shell was obtained.
num_compromised	Numeric	Compromised conditions observed.
attack_type	Categorical	Specific attack signature (neptune, smurf, mirai, zeus ...); 18 attack types.
attack_category	Target class	Normal, DoS, Probe, R2L, U2R or Botnet.

DS2 – Global Cyber Incidents (cyber_incidents.csv)

Table 2. Attributes of DS2 (16 columns).

Attribute	Type	Description
incident_id / date	Identifier	Incident identifier and reporting date (2019 – 2025).
target_country / sector	Categorical	Country and industry sector of the victim organisation.
network	Categorical	Network type that was breached.
attack_vector	Categorical	Primary technique (Ransomware, Phishing, DDoS, SQL Injection ...).
threat_actor	Categorical	Attributed adversary type.
vulnerability	Categorical	Root-cause weakness exploited.
defense_in_place	Categorical	Main defence the organisation had deployed.
affected_users / records_breached	Numeric	Users impacted and data records exposed.
data_exfiltrated_gb	Numeric	Volume of data stolen (GB).
detection_hours / resolution_hours	Numeric	Time to detect and time to contain and recover.
financial_loss_musd	Numeric	Estimated loss in US\$ million (regression target).
severity	Target class	Low, Medium, High or Critical.

Dataset Handling

The Datasets page shows each file's dimensions, missing cells, duplicate rows, a record preview, a data dictionary and a statistical summary. Both datasets can be downloaded as CSV or as ARFF, so the same data can be loaded into Weka to cross-check the J48 and Naive Bayes results. A third tab accepts a user upload (.csv, .xlsx or .xls, with a sheet picker for Excel files), which then appears in the dataset selector of every mining page. The record counts below are taken from the running application.

Table 3. Dataset size and data quality.

Dataset	Rows (raw)	Columns	Missing cells	Duplicates	Rows after cleaning
DS1 network traffic	30,180	23	1,050	180	30,000
DS2 cyber incidents	5,000	16	130	0	5,000

Table 4. Class distribution after cleaning.

Class	DS1 attack_category		Class	DS2 severity
Normal	13,631		Medium	2,117
DoS	5,850		Low	1,456
Probe	4,335		High	1,279
R2L	2,926		Critical	148
Botnet	2,495			
U2R	763			

DATA COLLECTION

Datasets

Two cyber-security datasets are bundled. You can also upload your own CSV or Excel file.

DS1 - Network Intrusion Traffic

DS2 - Global Cyber Incidents

Upload your own

Connection-level intrusion records modelled on the **NSL-KDD / KDD Cup '99** schema and captured on five networks (Corporate LAN, University campus, Cloud data-centre, IoT smart-grid, Banking core). Used for association rules, J48, Naive Bayes, regression and clustering.

Rows

30,180

Columns

23

Missing cells

1,050

Duplicate rows

180

	conn_id	timestamp	network	src_country	protocol	service	flag	duration	src_bytes	dst_bytes	packets	count	srv_count	error_rate	rem
0	C000001	2026-08-01 00:01:41	Banking-Core	Russia	tcp	rdp	SF	3.333	83	147	4	4	1	0.014	
1	C000002	2026-08-01 00:01:57	Corporate-LAN	China	tcp	rdp	S0	0	1	0	5	4	1	0.447	
2	C000003	2026-08-01 00:05:08	Cloud-DC	USA	tcp	http	S0	0	0	0	2	229	9	0.962	
3	C000004	2026-08-01 00:05:12	University-Campus	Iran	tcp	telnet	RSTO	5.01	36	44	4	2	1	0.009	
4	C000005	2026-08-01 00:06:35	University-Campus	India	tcp	https	REJ	0	4	0	2	25	3	0.128	
5	C000006	2026-08-01 00:07:13	Corporate-LAN	Russia	tcp	mqtt	REJ	0	6	0	3	4	1	0.482	
6	C000007	2026-08-01 00:11:38	Cloud-DC	India	tcp	http	S0	0	0	0	2	237	18	0.979	
7	C000008	2026-08-01 00:13:46	Banking-Core	Russia	udp	private	SF	0	28	0	1	52	52	0.035	

Figure 1. Datasets page showing DS1 dimensions, data quality counts and the record preview.

3. System Design and Technologies

The application follows a modular structure. `app.py` sets up the page and a top navigation bar with four groups (Overview, Warehouse, Mining, Evaluation). Each page lives in its own module in `views/`, and the shared logic for loading, cleaning, encoding, evaluation and styling lives in `core/`. Because of this separation, each technique can be changed or tested on its own without touching the others.

Table 5. Project modules.

Module	Responsibility
<code>app.py</code>	Page configuration, navigation (<code>st.navigation</code>) and footer.
<code>core/data.py</code>	Dataset loading and caching, cleaning, per-dataset defaults, discretisation, one-hot encoding, ARFF export.
<code>core/models.py</code>	Weka-style mixed Naive Bayes, evaluation (split / k-fold), per-class metrics, tree rules, gain ratio.
<code>core/ui.py</code>	Shared layout pieces (hero, KPI tiles, rule cards), colour palette and Plotly theme.
<code>views/dashboard.py, dataset.py</code>	Overview dashboard and dataset management / upload.
<code>views/warehouse.py, preprocessing.py</code>	Star schema, ETL and OLAP operations; preprocessing steps.
<code>views/association.py</code>	Apriori, FP-Growth, level-wise Apriori from scratch, per-network comparison, benchmark.
<code>views/j48.py, naive_bayes.py, common.py</code>	Classification pages and the shared evaluation display.
<code>views/regression.py, clustering.py</code>	Regression and clustering pages for both datasets.
<code>views/comparison.py</code>	Six-classifier comparison and best-model selection.
<code>data/generate_data.py</code>	Reproducible generator for DS1 and DS2.

Technologies Used

- **Python** – main programming language.
- **Streamlit** – web interface, navigation, widgets and page layout.
- **Pandas and NumPy** – data loading, cleaning, transformation and numerical work.
- **Scikit-learn** – decision tree, regression, clustering, PCA, metrics and the comparison classifiers.
- **MLxtend** – Apriori, FP-Growth and association-rule generation.
- **SciPy** – hierarchical linkage for the dendrogram.
- **Plotly** – interactive charts; **Graphviz** – decision-tree and star-schema diagrams.
- **openpyxl / xlrd** – reading uploaded Excel files.
- **GitHub** – source-code repository; **Streamlit Community Cloud** – hosting.

Application Flow

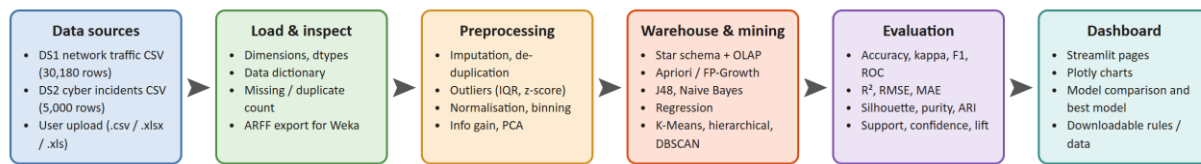


Figure 2. Application flow from data sources to the dashboard.

The Overview page summarises the main findings before the user goes into any one technique. **54.6%** of the 30,000 cleaned connections are malicious, and the attack mix differs sharply by network. IoT-Grid carries the largest malicious share (61.8%, mostly Botnet traffic), Cloud-DC is dominated by DoS (57.9%), University-Campus by Probe (56%), Corporate-LAN by DoS (50.2%) and Banking-Core by R2L credential attacks (48.2%). Attack volume peaks around 01:00 UTC, and a DoS flood campaign is visible on days 12 – 14. In DS2, the incidents add up to an estimated **US\$ 6,750 million** in losses. Healthcare has the highest average loss per incident, and ransomware is the costliest attack vector.

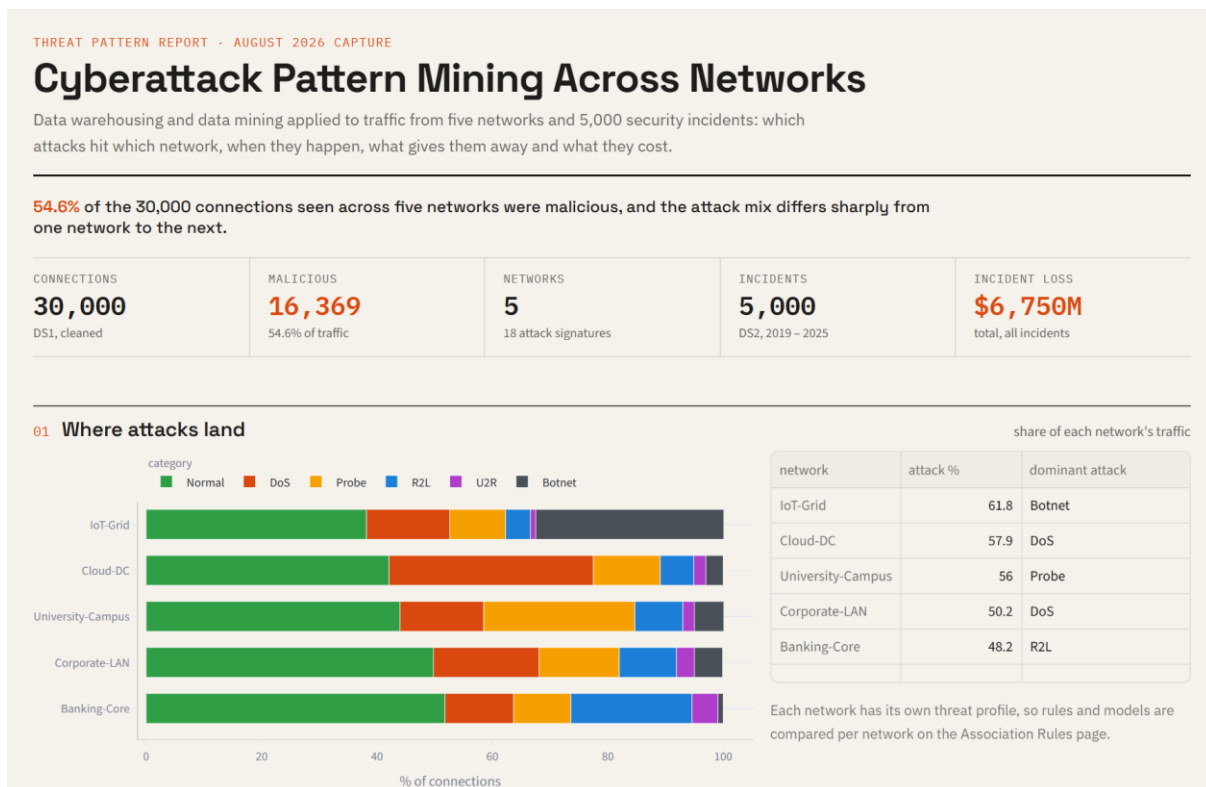


Figure 3. Overview dashboard: headline KPIs and the attack mix on each network.

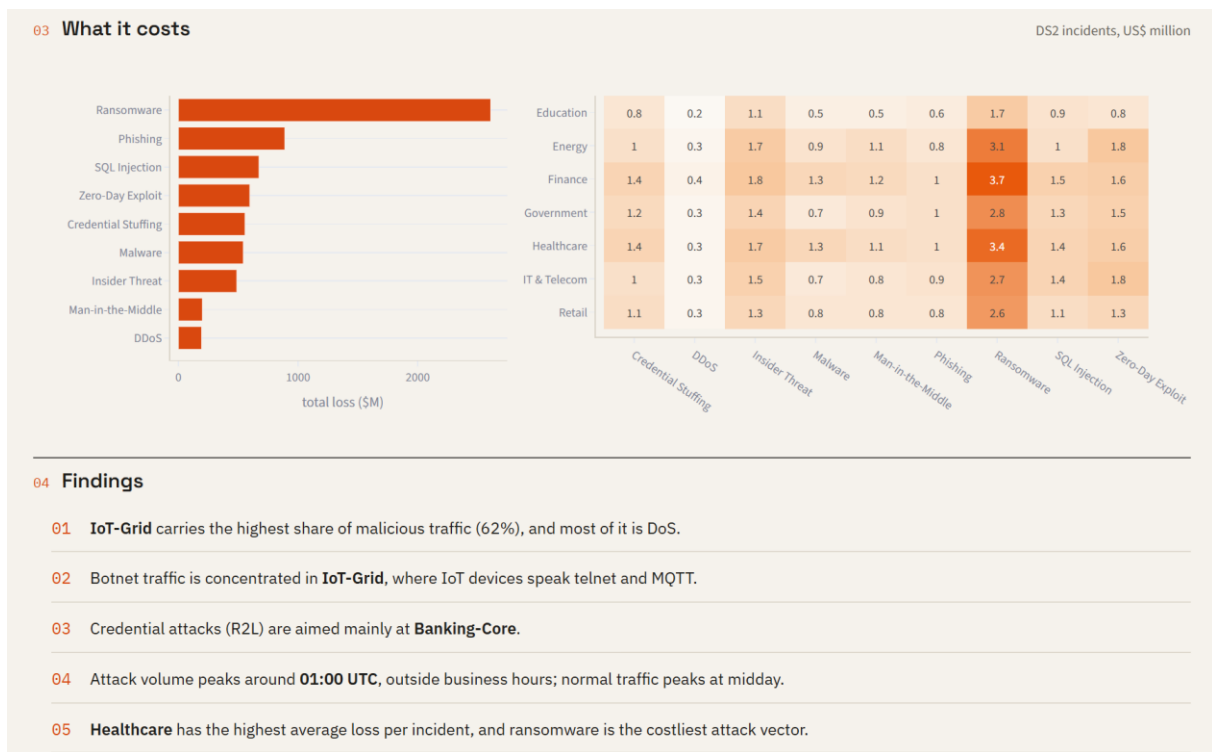


Figure 4. Overview dashboard: incident loss by attack vector and sector, and the automatic findings.

4. Data Warehouse and Data Preprocessing

Data Warehouse and OLAP

A data warehouse is a subject-oriented, integrated, time-variant and non-volatile collection of data that supports decision making. Here the subject is network attacks. During ETL, the raw DS1 connection logs are loaded into a star schema with one fact table and five dimension tables.

Table 6. Star schema produced by the ETL step.

Table	Rows	Contents
FACT_CONNECTION	30,000	Foreign keys + measures: duration, src_bytes, dst_bytes, packets, failed_logins, is_attack.
DIM_TIME	744	date, hour, day_of_week, week, month, is_weekend, time_of_day.
DIM_NETWORK	5	network, zone, criticality.
DIM_SOURCE	11	src_country, region.
DIM_SERVICE	56	service, protocol, flag.
DIM_ATTACK	19	attack_type, attack_category, kill_chain_stage.

- **Extract** – read the raw connection logs from the five network sensors (CSV).
- **Transform** – remove duplicates, impute missing bytes and durations, and derive hour, week, time of day, region, network zone, kill-chain stage and the is_attack flag.
- **Load** – generate a surrogate key for each dimension row; fact rows reference these keys.

The cube built on this schema supports all five OLAP operations. **Roll-up** climbs the time hierarchy (hour → day → week). At day level it shows the DoS flood on 12 – 14 August. **Drill-down** goes the other way (for example from attack category to individual attack type). **Slice** fixes one dimension, **dice** selects a sub-cube on two or more dimensions, and **pivot** rotates the cube into a cross-tab. For example, dicing on network ∈ {Cloud-DC, Banking-Core}, protocol = tcp and time of day ∈ {Night, Evening} gives a sub-cube of 4,566 connections. In it, DoS dominates Cloud-DC and R2L dominates Banking-Core.

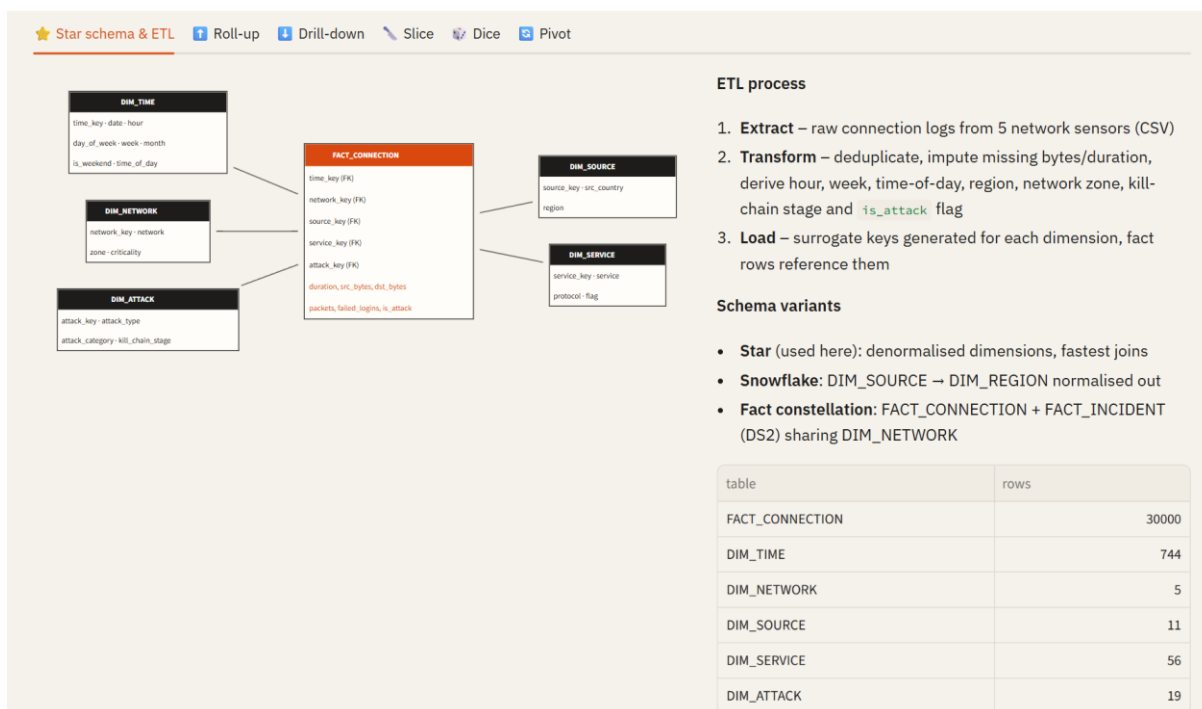


Figure 5. Star schema, ETL steps and the row count of each table.

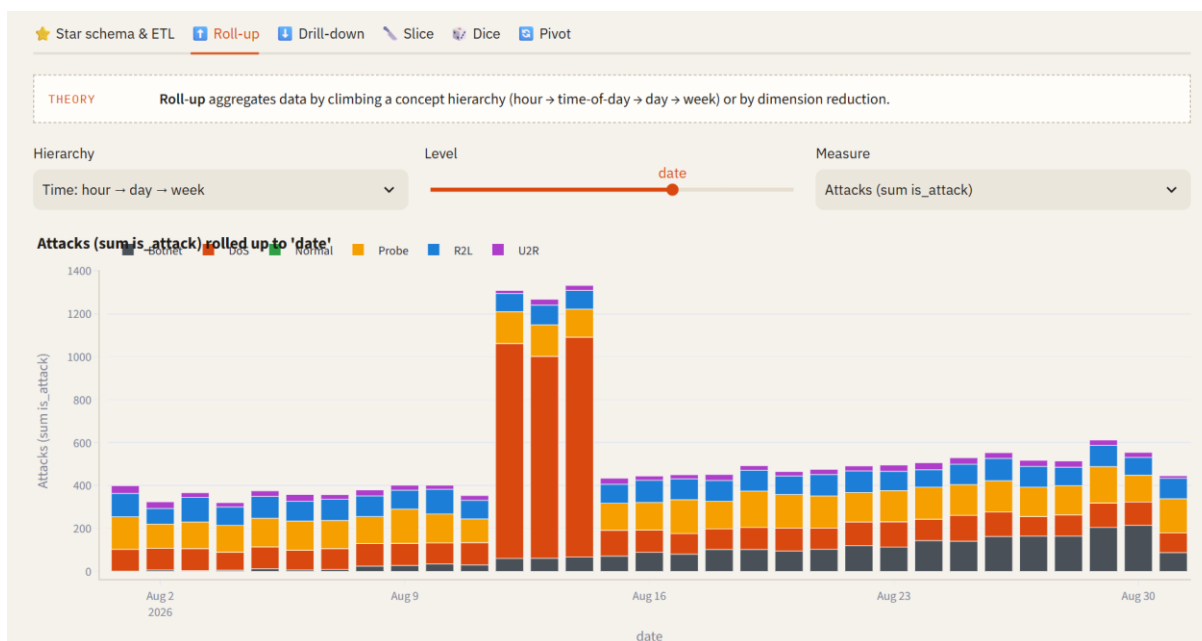


Figure 6. Roll-up of attacks along the time hierarchy (date level).

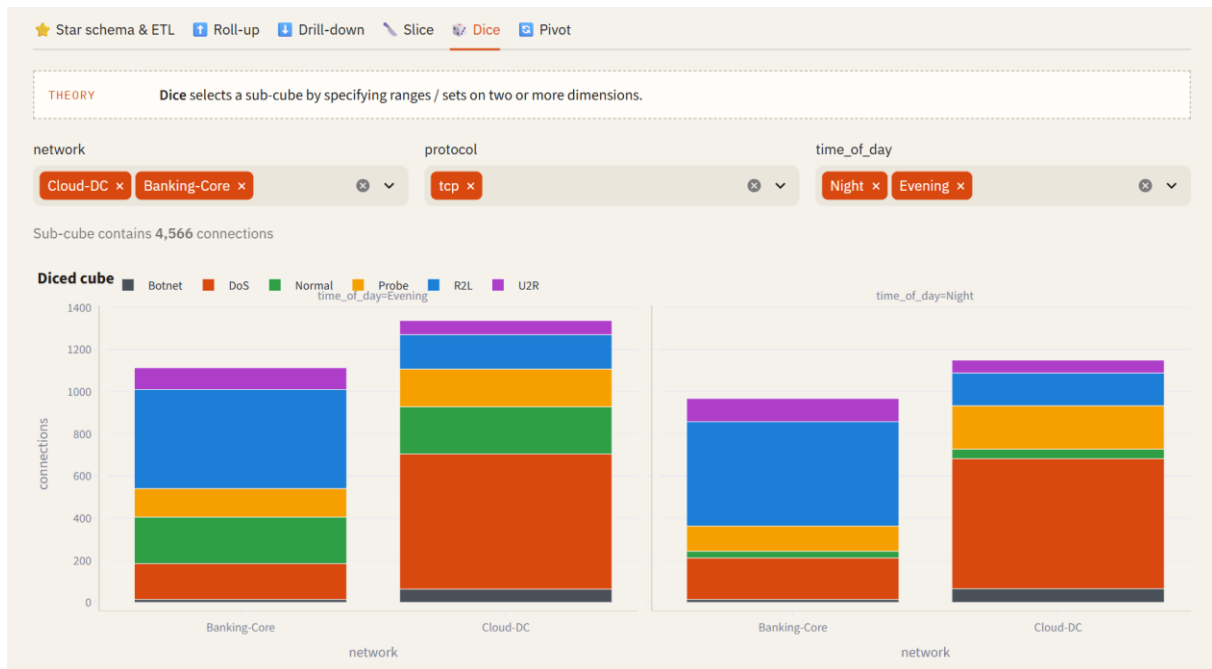


Figure 7. Dice operation on network, protocol and time of day.

Data Preprocessing

Preprocessing prepares the raw data for mining. The Preprocessing page works on any dataset and is organised in six tabs:

1. **Missing values and duplicates** – count missing cells per column, then impute numeric columns with the median or mean (or drop rows) and categorical columns with 'Unknown' or the mode, and remove duplicate rows.
2. **Outliers** – flag values outside $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ or with $|z| > 3$.
3. **Normalisation** – min-max scaling to $[0, 1]$, z-score standardisation and decimal scaling.
4. **Discretisation and smoothing** – equal-width and equal-frequency binning, and smoothing by bin means or bin boundaries (needed because Apriori works on categorical items).
5. **Feature selection** – Pearson correlation matrix and mutual information (information gain) with the class.
6. **PCA** – projection of the standardised attributes onto principal components, with a scree plot.

On DS1, the 1,050 missing cells are in dst_bytes (442), src_country (320) and duration (288). Median imputation, 'Unknown' for the categorical column and duplicate removal reduce the data from 30,180 to 30,000 rows with no missing values. The same cleaning is applied automatically

before every mining page. On `src_bytes`, the IQR rule flags 2,427 rows (8.1%) and the z-score rule flags about 1%. Many of these outliers are the attacks themselves (floods and data exfiltration), so they are kept rather than deleted. PCA shows that 9 of the 14 numeric components retain 95% of the variance, and that the attack classes form clearly separated groups in the first two components.

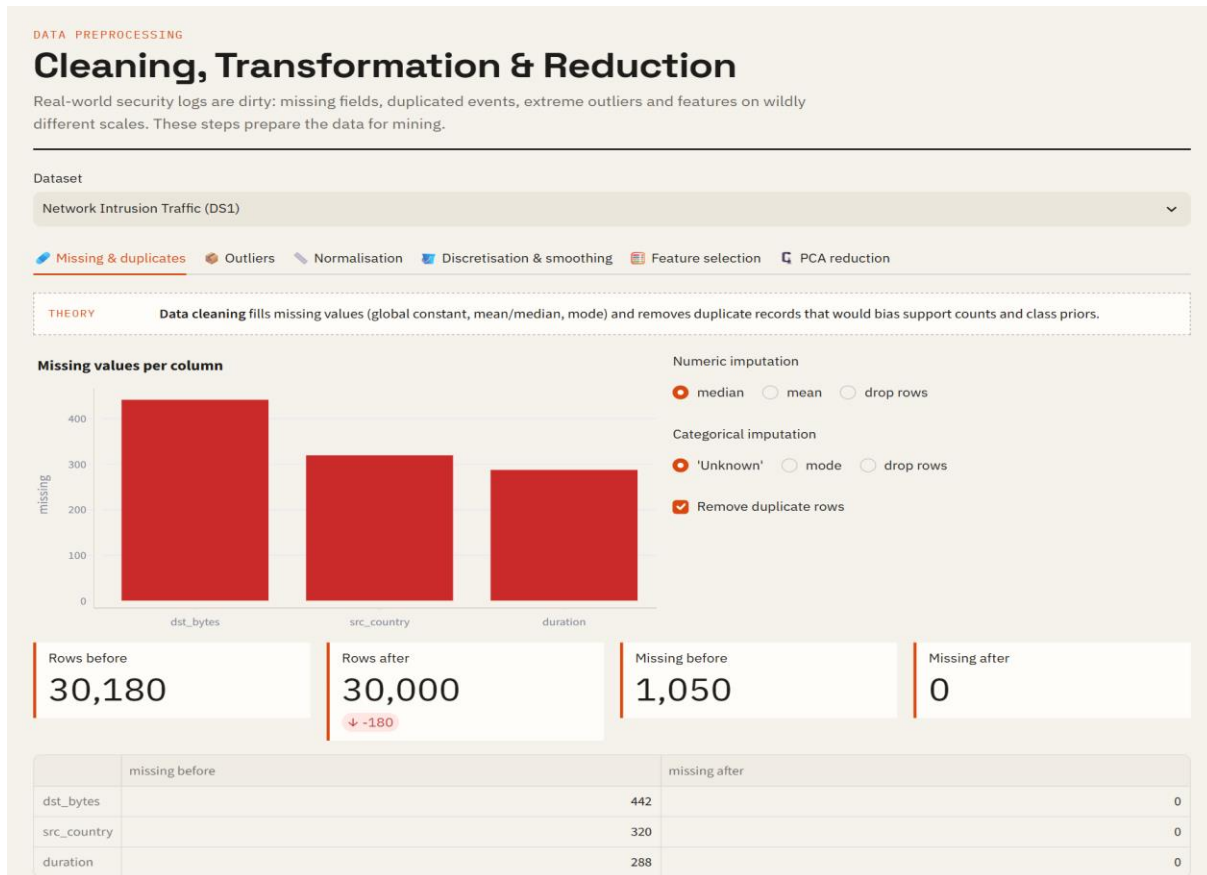


Figure 8. Missing-value counts and cleaning results (rows and missing cells before and after).

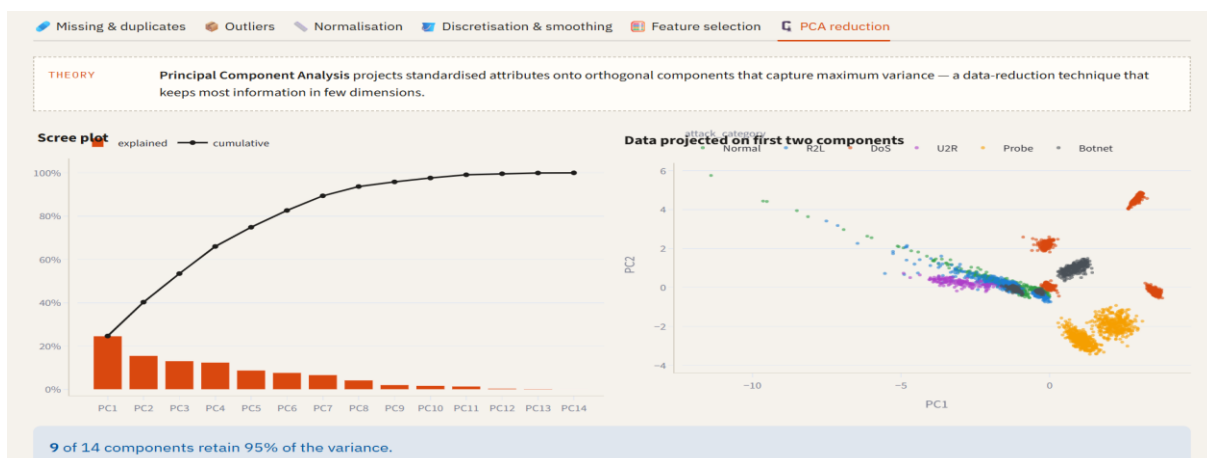


Figure 9. PCA scree plot and DS1 projected onto the first two principal components.

5. Association Rule Mining

Association rule mining is an unsupervised technique that finds items which occur together. Each DS1 connection is treated as a market-basket transaction of attribute=value items such as protocol=tcp, flag=S0 or count=high. Numeric attributes are first discretised into three equal-frequency bins (low / med / high). Rules whose consequent is attack_category=... are attack signatures.

Key Measures

- **Support** ($A \Rightarrow B$) = $P(A \cup B)$: the share of transactions that contain both sides.
- **Confidence** ($A \Rightarrow B$) = $\text{sup}(A \cup B) / \text{sup}(A)$: how often the rule is right when A occurs.
- **Lift** = confidence / $\text{sup}(B)$: a value above 1 means a positive correlation, but not causation.
- **Apriori property**: every subset of a frequent itemset is frequent, so any candidate with an infrequent subset is pruned. **FP-Growth** avoids candidate generation by compressing the data into an FP-tree.

Settings and Results

Nine attributes (network, src_country, protocol, service, flag, src_bytes, count, serror_rate and attack_category) give **51 distinct items** over **30,000 transactions**. With minimum support 0.02, minimum confidence 0.60, minimum lift 1.2 and itemsets of up to 4 items, FP-Growth finds **2,551 frequent itemsets** and **3,317 strong rules**. 585 of these rules predict an attack category. The strongest signatures are:

Table 7. Top attack-signature rules on DS1 (all networks).

Rule (IF ... THEN ...)	Support	Confidence	Lift
count=low, service=ftp, src_bytes=high $\Rightarrow$ R2L	0.027	0.84	8.66
count=low, flag=SF, service=ftp $\Rightarrow$ R2L	0.035	0.80	8.17
count=high, protocol=tcp, service=telnet $\Rightarrow$ Botnet	0.022	0.67	8.09
protocol=icmp, src_bytes=low $\Rightarrow$ Probe	0.046	1.00	6.92
protocol=icmp, serror_rate=high, src_bytes=low $\Rightarrow$ Probe	0.044	1.00	6.92
count=high, protocol=icmp, src_bytes=high $\Rightarrow$ DoS	0.050	1.00	5.13
protocol=udp, service=dns, src_bytes=high $\Rightarrow$ Normal	0.028	1.00	2.20

These rules match known attack behaviour. R2L password and warez attacks send large payloads over a few FTP sessions, Mirai-style botnets open many telnet connections, Probe scans are small ICMP packets with SYN errors, and DoS floods are bursts of large ICMP packets.

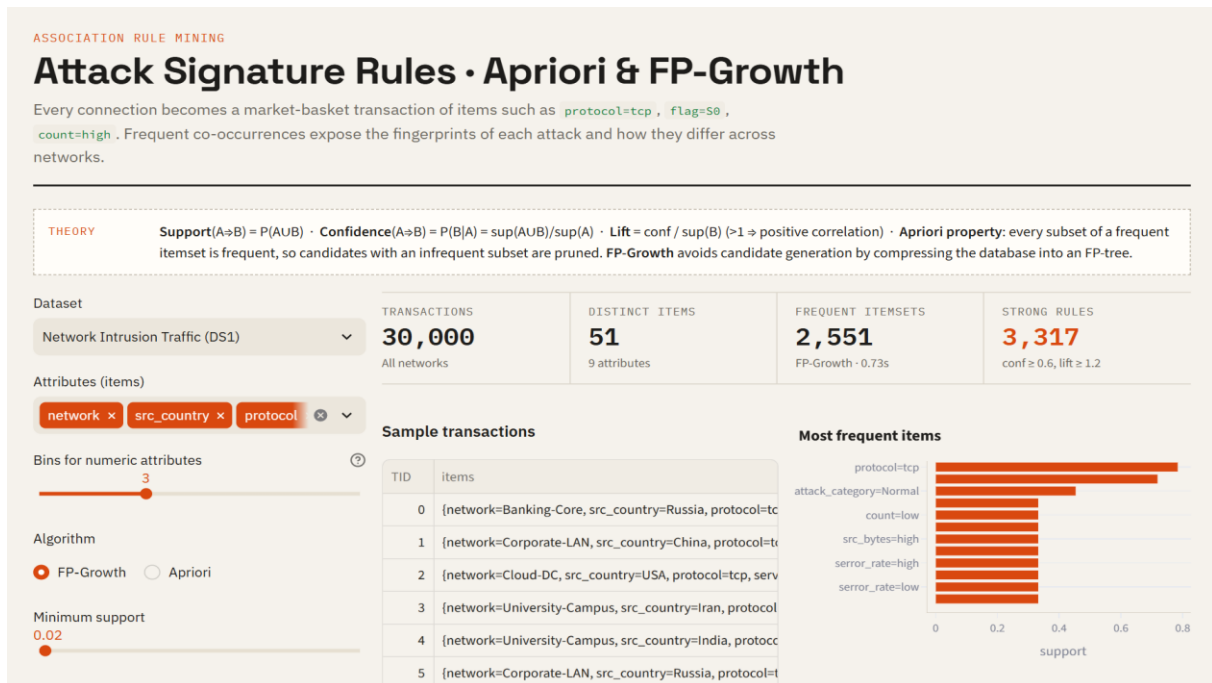


Figure 10. Association Rules page: parameters, transaction summary and the most frequent items.

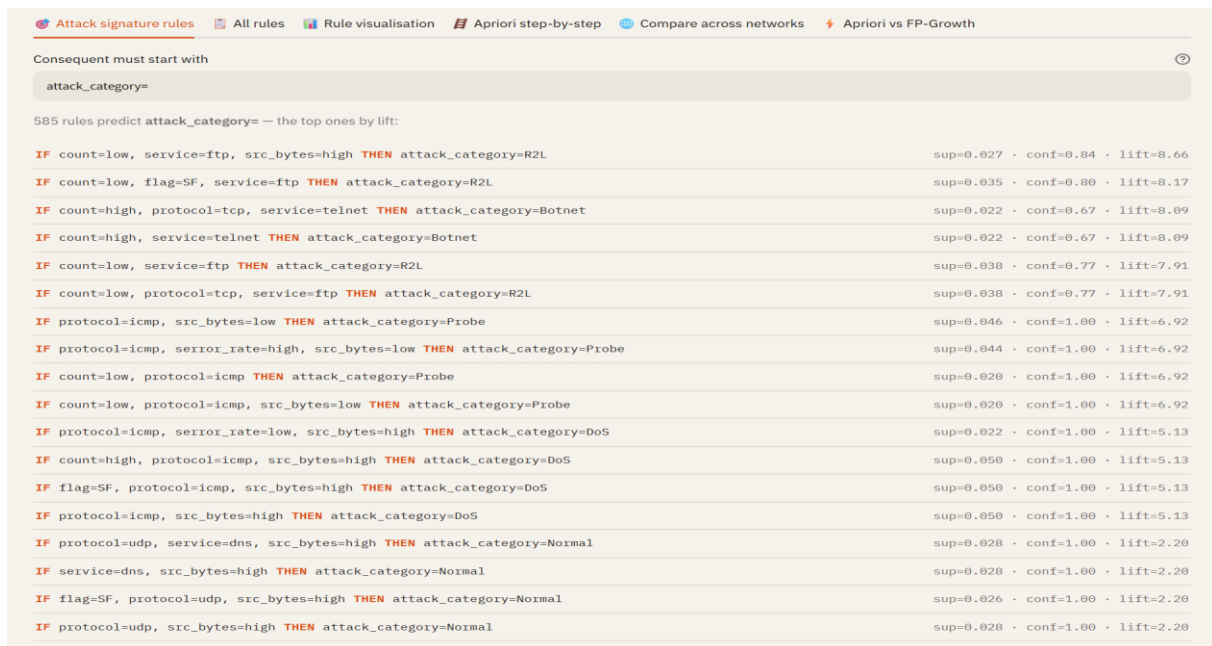


Figure 11. Attack-signature rules ranked by lift.

Apriori Step by Step

The app also contains a from-scratch level-wise Apriori, which records the candidate itemsets C_k and frequent itemsets L_k at each level. From level 3 onwards, the Apriori property removes most of the candidates before the database is scanned.

Table 8. Level-wise Apriori on DS1 (min support 0.02).

Level k	C_k generated	Pruned by Apriori property	L_k frequent
1	51	0	49
2	1,019	0	449
3	5,326	3,186	1,036
4	7,314	6,015	1,017

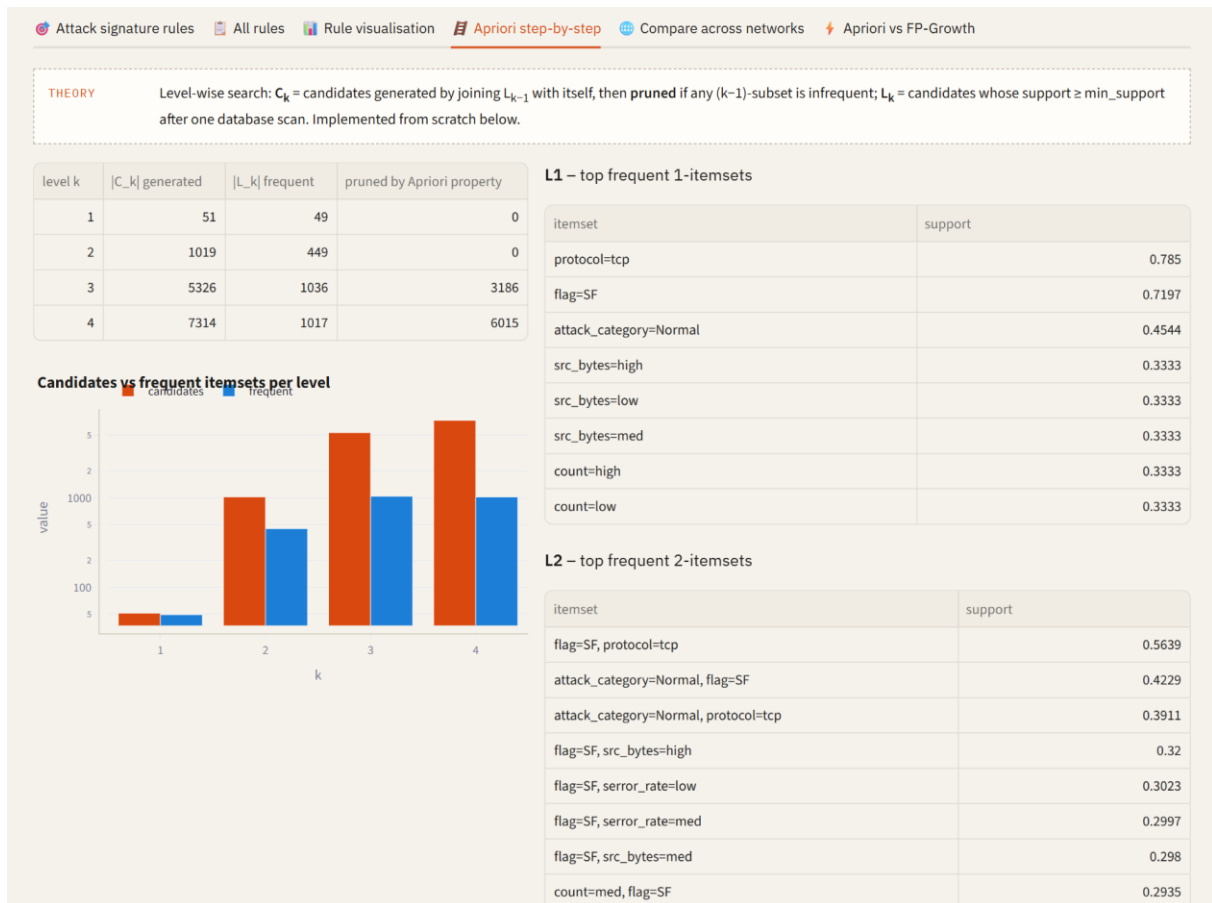


Figure 12. Apriori step-by-step: candidates versus frequent itemsets and the top L1 – L4 itemsets.

Comparison Across Networks and Algorithms

Rules are also mined separately inside each network and their confidence compared. Some signatures are universal (ICMP probes), while others occur on only one network: count=med, flag=REJ $\Rightarrow$ Probe only on Cloud-DC (lift 6.79); count=low, flag=RSTO $\Rightarrow$ R2L only on Banking-Core (lift 3.80); service=private, src_bytes=med $\Rightarrow$ Botnet only on IoT-Grid (lift 3.07). Defences therefore need to be tuned per network.

Apriori and FP-Growth return identical frequent itemsets. In the benchmark (mlxtend implementations), Apriori was faster at every support level tested: 0.03 s against 0.22 s at support 0.2, and 0.28 s against 0.44 s at support 0.03 (1,490 itemsets). With only 51 items and itemsets capped at four, Apriori's candidate sets stay small, while FP-Growth pays a fixed cost to build the tree. The gap narrows as support falls, which is where FP-Growth normally takes the lead.

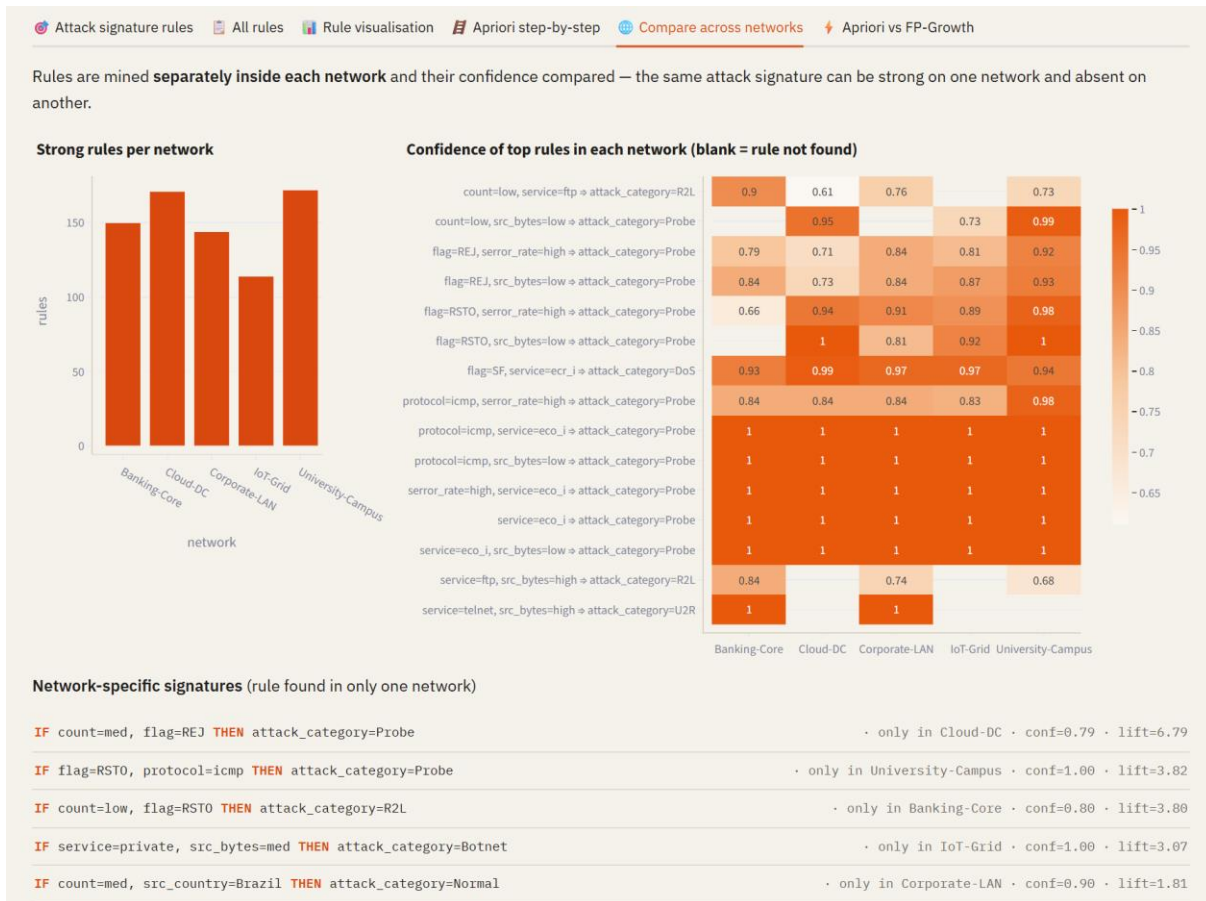


Figure 13. Strong rules per network and the confidence of the top rules in each network.

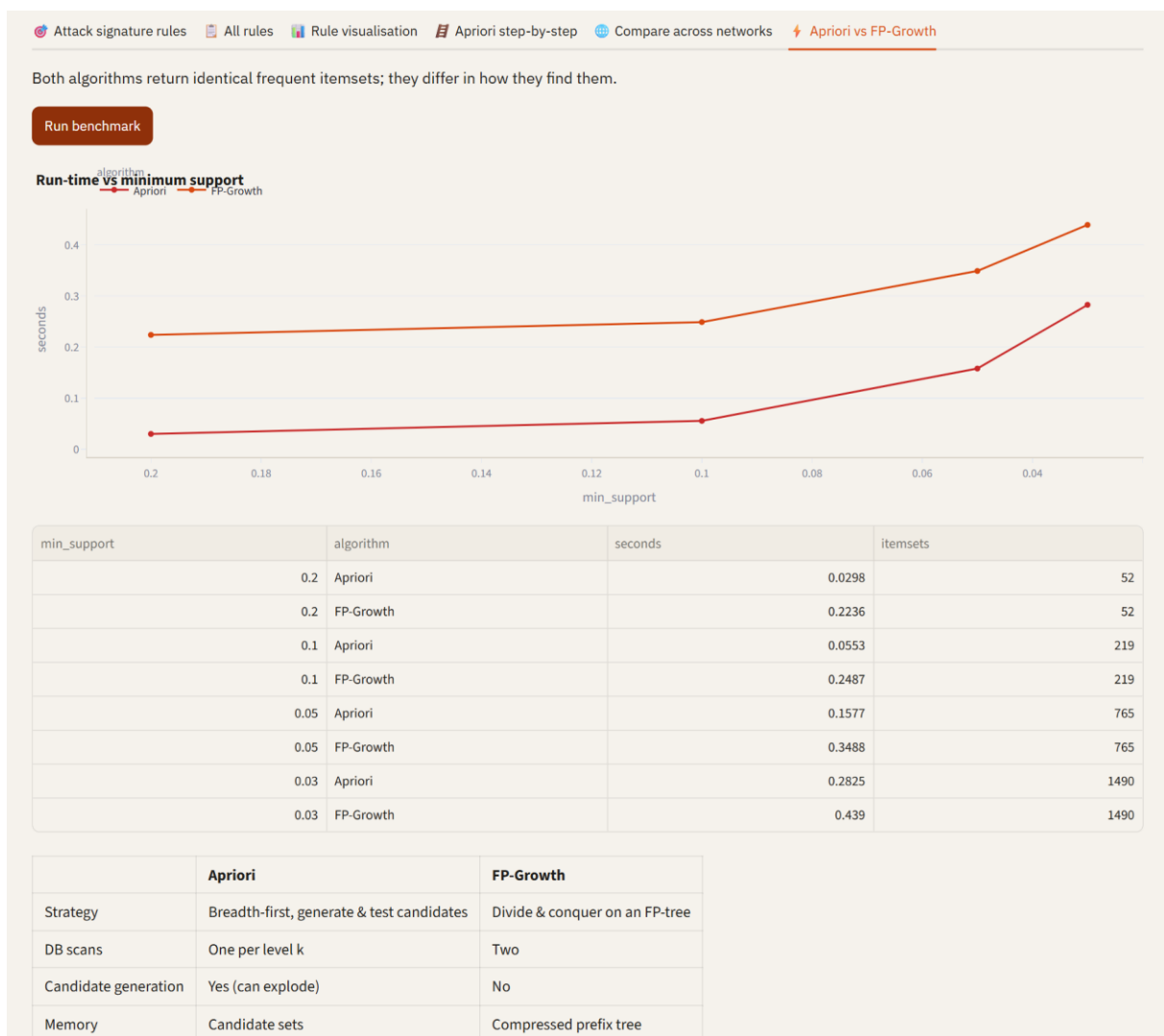


Figure 14. Apriori versus FP-Growth run time at decreasing minimum support.

6. Classification: J48 Decision Tree and Naive Bayes

Both classifiers predict **attack_category** on DS1 (six classes) and **severity** on DS2 (four classes). They use the same 16 DS1 attributes (protocol, service, flag, duration, byte counts, connection rates, error rates, login and compromise indicators) and the same 66% / 34% stratified split (random_state 42). The attack_type column is excluded because it would leak the label. The pages also offer 10-fold cross-validation.

Decision Tree / J48-Equivalent Classification

J48 is Weka's implementation of Quinlan's C4.5. This project does not call Weka; it uses scikit-learn's DecisionTreeClassifier configured to behave like J48: entropy (information gain) as the split criterion, min_samples_leaf = 2 (Weka's -M 2) and cost-complexity pruning ($\alpha = 0.0005$) in place of C4.5's confidence-factor pruning. It should therefore be described as a **J48-equivalent** tree. To cross-check, the page exports the data as ARFF for Weka, prints Weka-style run information and per-class accuracy, converts the tree into IF-THEN rules and shows a C4.5 gain-ratio table for the candidate root attributes.

On DS1 the tree has **41 leaves, 81 nodes and depth 10**. It reaches **99.47% accuracy** (10,146 of 10,200 test connections correct), **kappa 0.993**, weighted **F1 0.995** and **ROC area 0.999**. The largest rules read naturally. For example, IF duration ≤ 0.0005 AND error_rate ≤ 0.171 AND count > 169 THEN DoS covers 2,865 training connections with 100% confidence, and IF duration ≤ 0.0005 AND error_rate > 0.203 THEN Probe covers 2,839. The gain-ratio table ranks root_shell first (0.826), but scikit-learn chooses binary splits by information gain, so the fitted tree uses duration at the root.

Table 9. J48 detailed accuracy by class (DS1, test set).

Class	TP rate	FP rate	Precision	Recall	F-measure	ROC area	Support
Botnet	0.994	0.002	0.975	0.994	0.984	1.000	848
DoS	0.999	0.000	1.000	0.999	0.999	0.999	1,989
Normal	0.995	0.004	0.996	0.995	0.995	0.998	4,635
Probe	0.995	0.000	1.000	0.995	0.998	0.998	1,474
R2L	0.987	0.001	0.988	0.987	0.987	0.997	995
U2R	0.988	0.000	1.000	0.988	0.994	1.000	259

J48 (C4.5) Decision Tree

J48 is Weka's implementation of Quinlan's C4.5: a tree grown top-down by choosing the attribute with the highest information-gain ratio at each node, then pruned to avoid over-fitting.

THEORY

$\text{Info}(D) = -\sum p_i \log_2 p_i$ · $\text{Gain}(A) = \text{Info}(D) - \sum |D_j|/|D| \cdot \text{Info}(D_j)$ · $\text{SplitInfo}(A) = -\sum |D_j|/|D| \log_2(|D_j|/|D|)$ · $\text{GainRatio} = \text{Gain} / \text{SplitInfo}$. Weka parameters: **-c** confidence factor (pruning), **-M** minimum instances per leaf.

Dataset

Network Intrusion Traffic (DS1)

=== Run information === · dataset Network Intrusion Traffic (DS1) · 30,000 instances · 16 attributes · test mode: 66% train / 34% test split
Number of leaves: 41 · Size of the tree: 81 · Depth: 10

Class attribute

attack_category

ACCURACY

99.47%

10,146 correctly classified

KAPPA

0.993

agreement beyond chance

WEIGHTED F1

0.995

P=0.995 · R=0.995

ROC AREA

0.999

one-vs-rest, weighted

Attributes

protocol × service × flag × dir ×

Test options

☒ Percentage split☐ 10-fold cross-validation

Training %

66

J48 parameters

-M min instances per leaf

2

Max depth (0 = unlimited)

0

Pruning strength (cost-complexity α)

0.0005

Confusion matrix · J48

	predicted					
	Botnet	DoS	Normal	Probe	R2L	U2R
Botnet	843	0	5	0	0	0
DoS	2	1987	0	0	0	0
Normal	16	0	4611	0	8	0
Probe	4	0	2	1467	1	0
R2L	0	0	13	0	982	0
U2R	0	0	0	0	3	256

Detailed accuracy by class

class	TP rate	FP rate	precision	recall	F-measure	ROC area
Botnet	0.994	0.002	0.975	0.994	0.984	
DoS	0.999	0	1	0.999	0.999	0.99
Normal	0.995	0.004	0.996	0.995	0.995	0.99
Probe	0.995	0	1	0.995	0.998	0.99
R2L	0.987	0.001	0.988	0.987	0.987	0.99
U2R	0.988	0	1	0.988	0.994	

F-measure per class

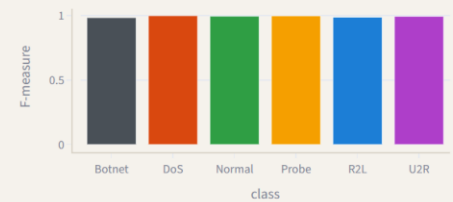


Figure 15. J48 page: run information, evaluation metrics, confusion matrix and per-class accuracy.

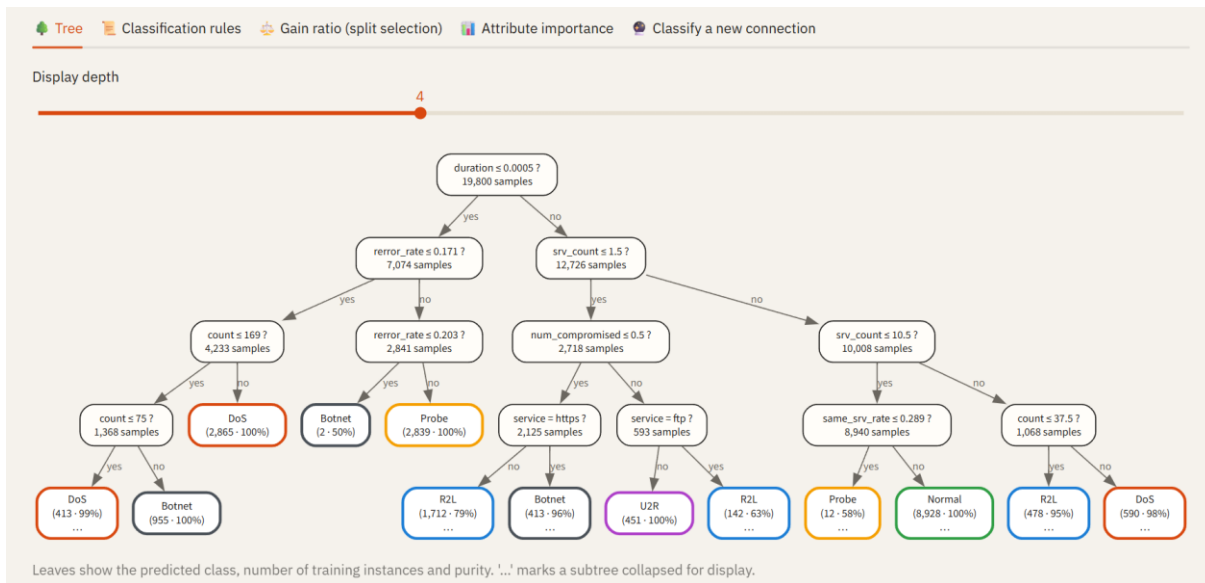


Figure 16. Fitted J48 tree (top four levels) with the class, size and purity of each leaf.

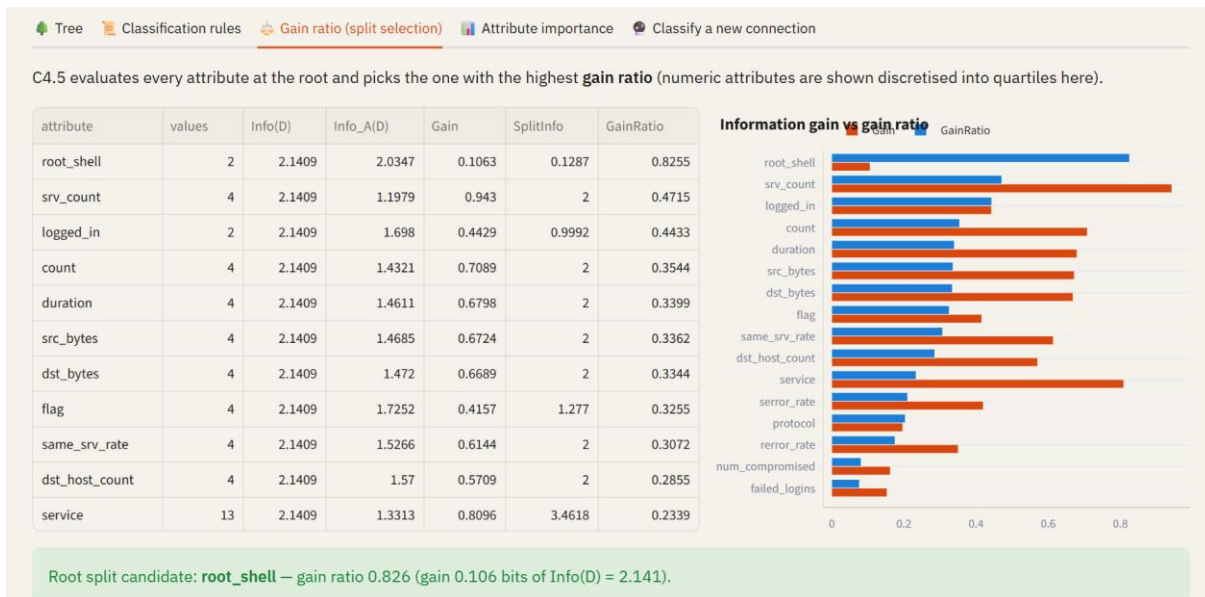


Figure 17. C4.5 information gain and gain ratio of each attribute at the root.

Naive Bayes Classification

Naive Bayes applies Bayes' theorem, $P(C | X) \propto P(C) \cdot \prod P(x_k | C)$, and assumes that the attributes are conditionally independent given the class. The project uses a custom MixedNB class that mirrors `weka.classifiers.bayes.NaiveBayes`. Numeric attributes use a Gaussian likelihood per class, after $\log(1 + x)$ on heavy-tailed byte counts. Nominal attributes use frequency tables with

Laplace correction $(\text{count} + 1) / (n + |\text{values}|)$. The page shows the class priors, the likelihood tables and a step-by-step posterior calculation for any record, computed in log-space to avoid underflow.

On DS1 (13 Gaussian + 3 nominal attributes), Naive Bayes reaches **87.41% accuracy**, **kappa 0.822**, weighted **F1 0.862** and **ROC area 0.989**. It is perfect on Probe and strong on Normal (F1 0.948) and DoS (0.900), but weak on Botnet (F1 0.371), R2L (0.656) and U2R (0.657). These classes are defined by combinations of correlated features (count, srv_count, error_rate), which breaks the independence assumption. The class priors are Normal 0.454, DoS 0.195, Probe 0.145, R2L 0.098, Botnet 0.083 and U2R 0.026.

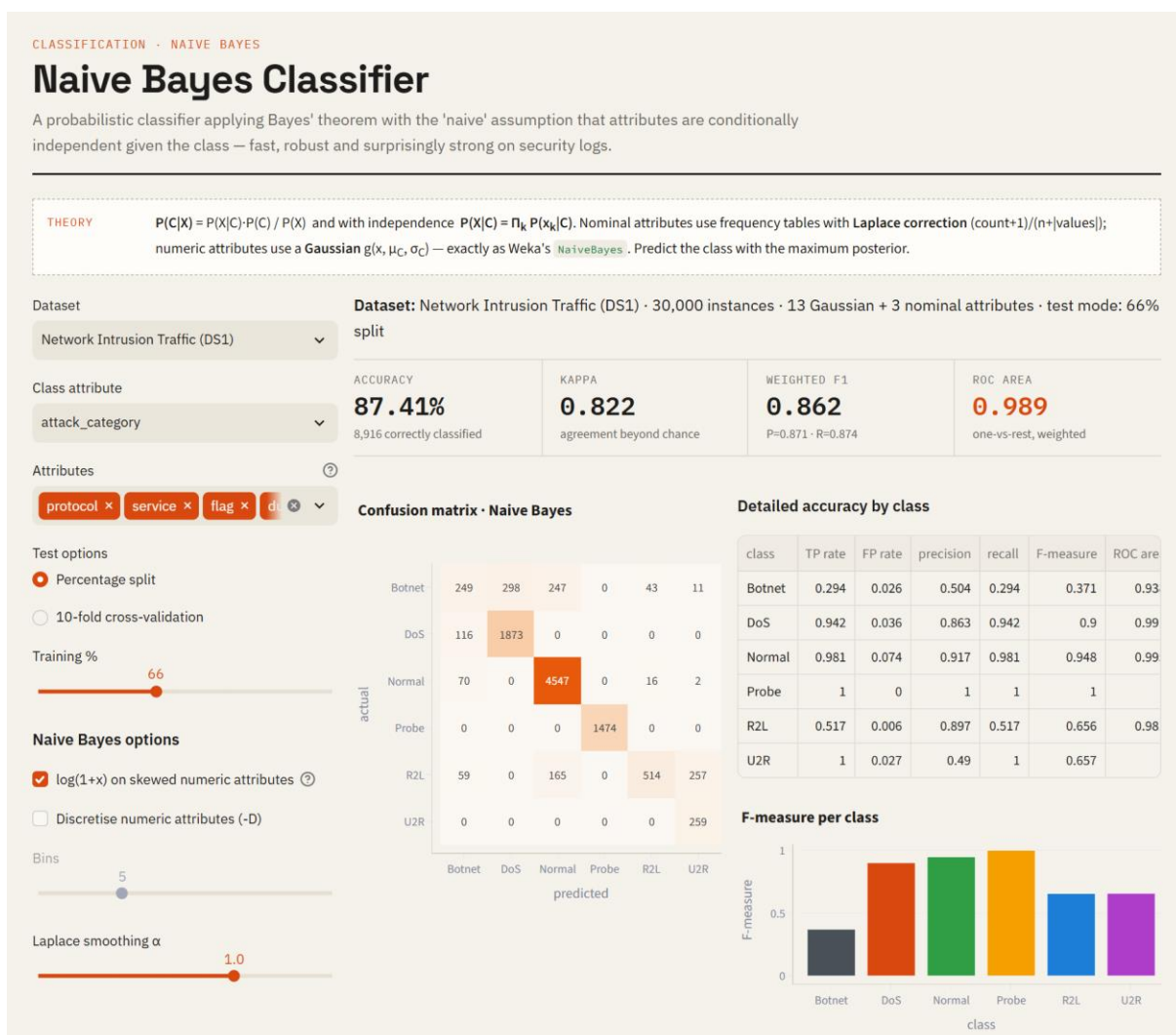


Figure 18. Naive Bayes page: evaluation metrics, confusion matrix and per-class accuracy.

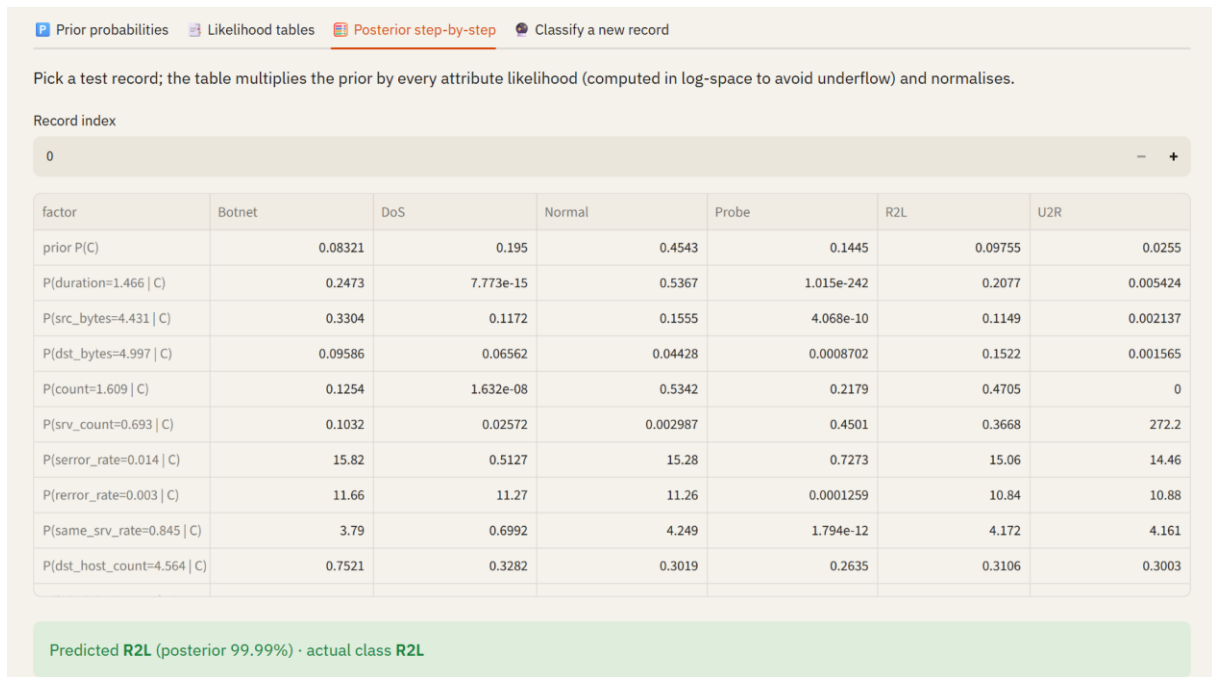


Figure 19. Posterior calculated step by step: the prior multiplied by each attribute likelihood.

Evaluation

Both classifiers are evaluated with the same split and the same target. Accuracy, Cohen's kappa, weighted precision, recall and F1, and one-vs-rest ROC area are reported, and the confusion matrix shows which classes are confused.

Table 10. J48 versus Naive Bayes on both datasets (66% split).

Dataset	Model	Accuracy	Kappa	Weighted F1	ROC area
DS1 (attack_category)	J48	99.47%	0.993	0.995	0.999
DS1 (attack_category)	Naive Bayes	87.41%	0.822	0.862	0.989
DS2 (severity)	J48	76.76%	0.655	0.768	0.851
DS2 (severity)	Naive Bayes	74.35%	0.603	0.737	0.894

J48 is clearly better on DS1 because it captures interactions between attributes. On DS2 the gap is small, and Naive Bayes even has the higher ROC area (0.894 against 0.851). This means its probability ranking is good even when the top-1 prediction is wrong. However, Naive Bayes finds only 16% of the rare Critical incidents, against 70% for J48.

7. Regression and Clustering

Regression

Regression predicts a continuous value. The Regression page offers simple linear, multiple linear, polynomial and ridge (regularised) regression on both datasets. It reports R^2 , adjusted R^2 , RMSE and MAE on a 30% held-out test set, with actual-versus-predicted and residual plots, the fitted equation and a least-squares working of β_0 and β_1 . The tables below use multiple linear regression.

Table 11. Multiple linear regression results.

Dataset / target	Predictors	R^2 (test)	Adj. R^2	RMS E	MAE
DS1 – packets	src_bytes, dst_bytes, duration, count	0.8861	0.8860	11.212	3.563
DS2 – financial_loss_musd	records_breached, resolution_hours, affected_users, detection_hours	0.7470	0.7463	0.815	0.480

DS1 equation: $\text{packets} = 2.741 + 0.001161 \cdot \text{src_bytes} + 0.001271 \cdot \text{dst_bytes} + 0.003412 \cdot \text{duration} - 0.001033 \cdot \text{count}$. The four predictors explain 88.6% of the variance in packet count, and test R^2 is higher than train R^2 (0.850), so the model is not over-fitting. A simple regression on src_bytes alone explains only 8.6% ($r = 0.293$), which shows the value of adding predictors.

DS2 equation: $\text{financial_loss} = 0.2274 + 4.777 \times 10^{-6} \cdot \text{records_breached} + 0.01209 \cdot \text{resolution_hours} + 1.39 \times 10^{-6} \cdot \text{affected_users} - 0.003905 \cdot \text{detection_hours}$. The model explains 74.7% of the variance in incident loss, with a mean absolute error of US\$ 0.48 million. Records breached on their own already give $r = 0.745$ ($r^2 = 0.555$), and resolution time adds most of the rest: each extra hour of recovery adds about US\$ 12,000.

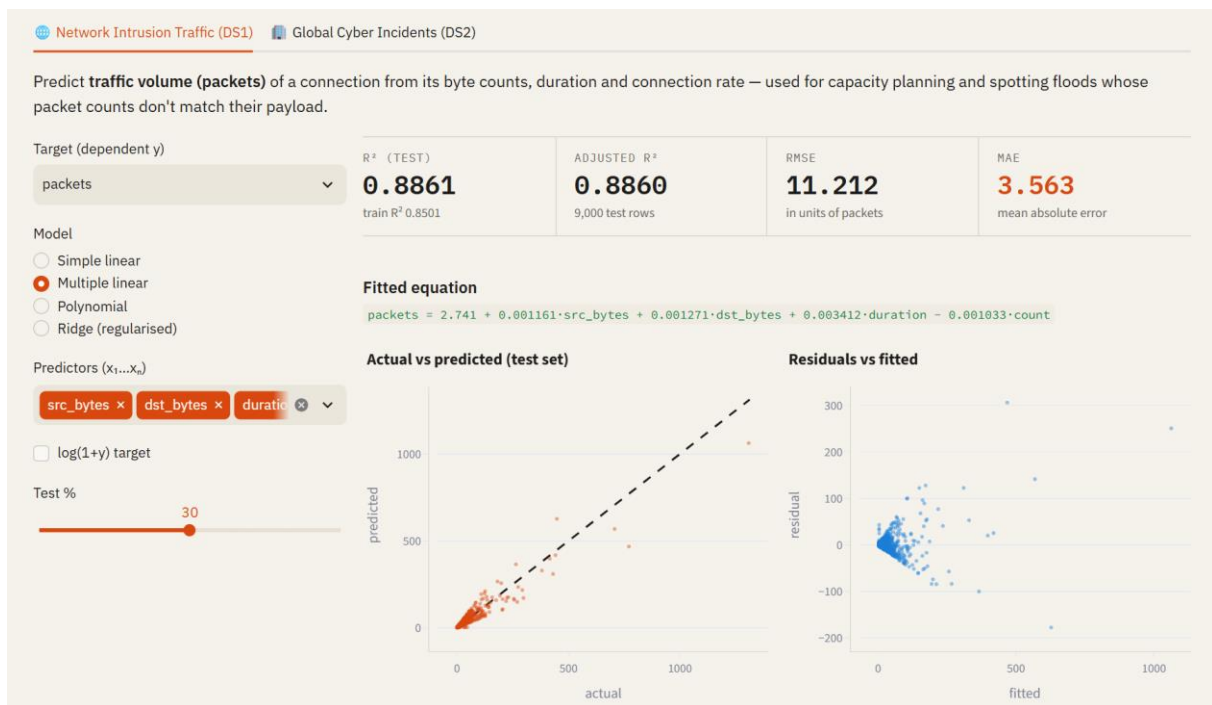


Figure 20. Regression on DS1 (packets): metrics, fitted equation, actual-vs-predicted and residual plots.

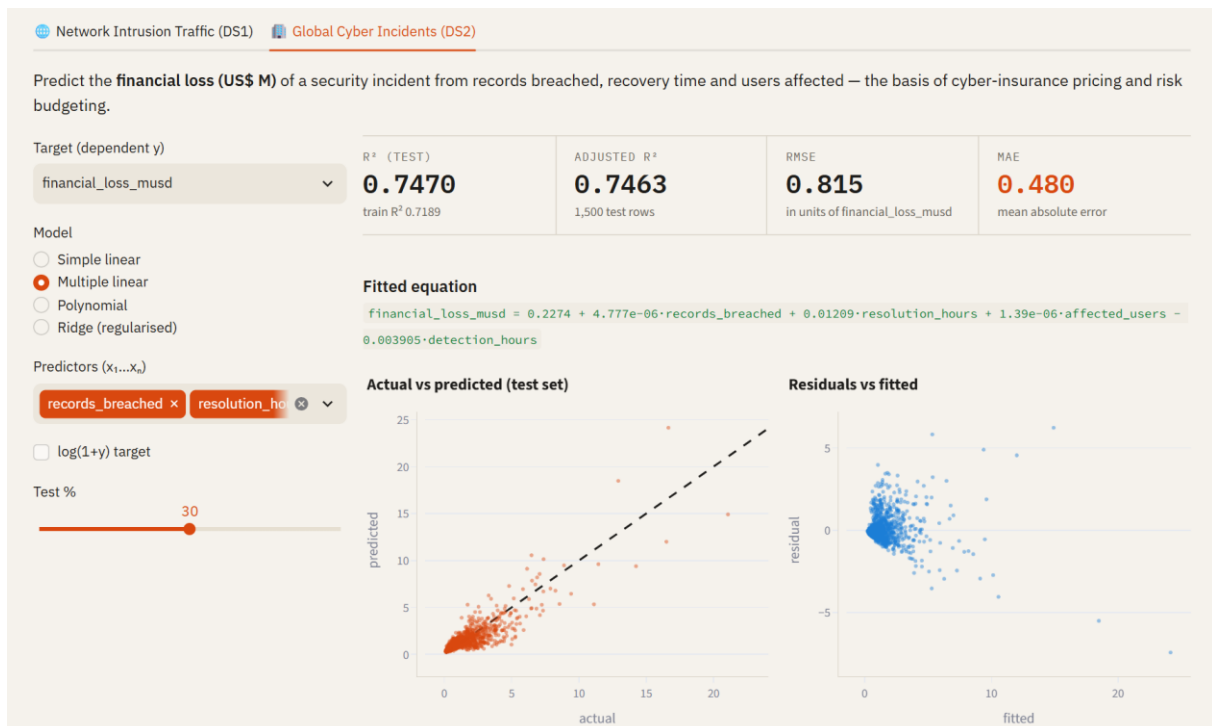


Figure 21. Regression on DS2 (financial loss): metrics, fitted equation and diagnostic plots.

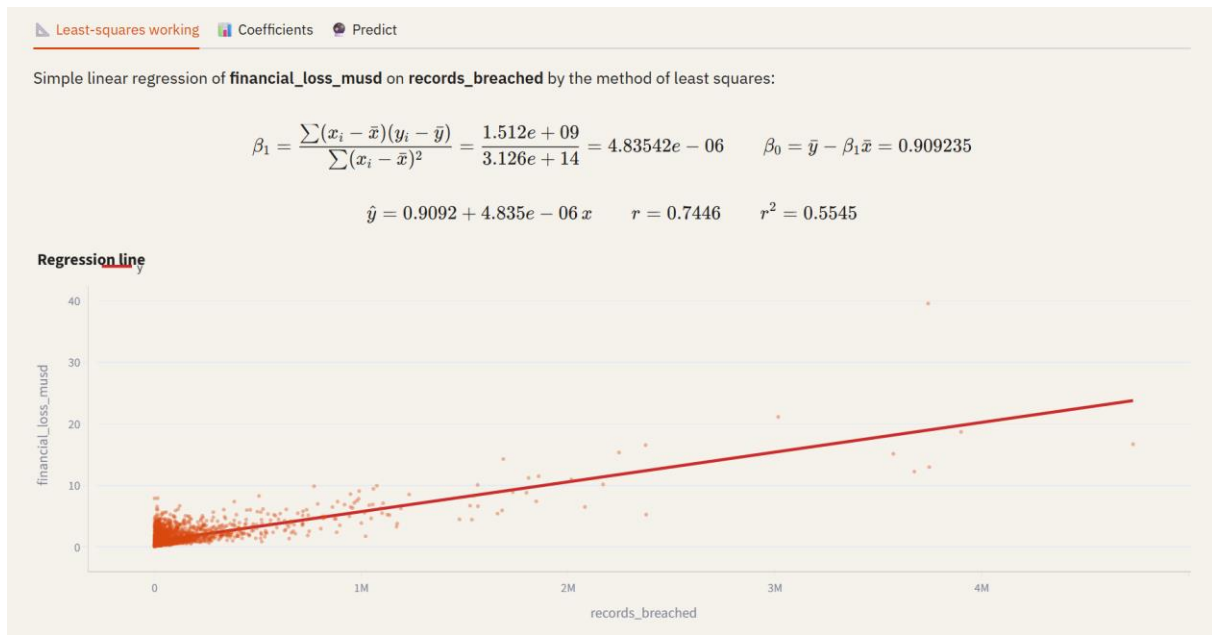


Figure 22. Least-squares working and regression line for loss against records breached.

Clustering

Clustering groups records without using the labels. The Clustering page offers **K-Means** (with elbow and silhouette plots for choosing k), **agglomerative hierarchical clustering** (ward, complete, average or single linkage, with a dendrogram) and **DBSCAN** (with a k -distance plot for choosing ϵ ; noise points are listed as potential anomalies). Skewed attributes are log-transformed and every attribute is z-score scaled, because all three methods are distance-based. The discovered clusters are then compared with the true labels using purity, the adjusted Rand index (ARI) and normalised mutual information (NMI).

Table 12. Clustering results.

Dataset / method	k	Silhouette	Davies-Bouldin	ARI	NMI	Purity
DS1 – K-Means (8 behaviour attributes)	5	0.663	0.649	0.571	0.670	79.43%
DS1 – Hierarchical, ward linkage	4	0.649	0.629	0.595	–	–
DS2 – K-Means (5 impact attributes)	4	0.227	1.401	0.191	0.294	65.46%

On DS1, the elbow curve flattens at $k = 5$, which is also where the silhouette peaks (0.66) before dropping to 0.41 at $k = 6$. Without seeing any labels, K-Means separates the attacks well: two pure

Probe clusters, one pure DoS cluster and one DoS-and-Botnet cluster. The remaining cluster mixes Normal traffic with the stealthier R2L and U2R attacks, which look like ordinary sessions in volume and rate features. The incidents in DS2 are much less separable (silhouette 0.23). Impact grows continuously rather than in distinct tiers, although one cluster still gathers almost all Critical incidents (147 of 148).

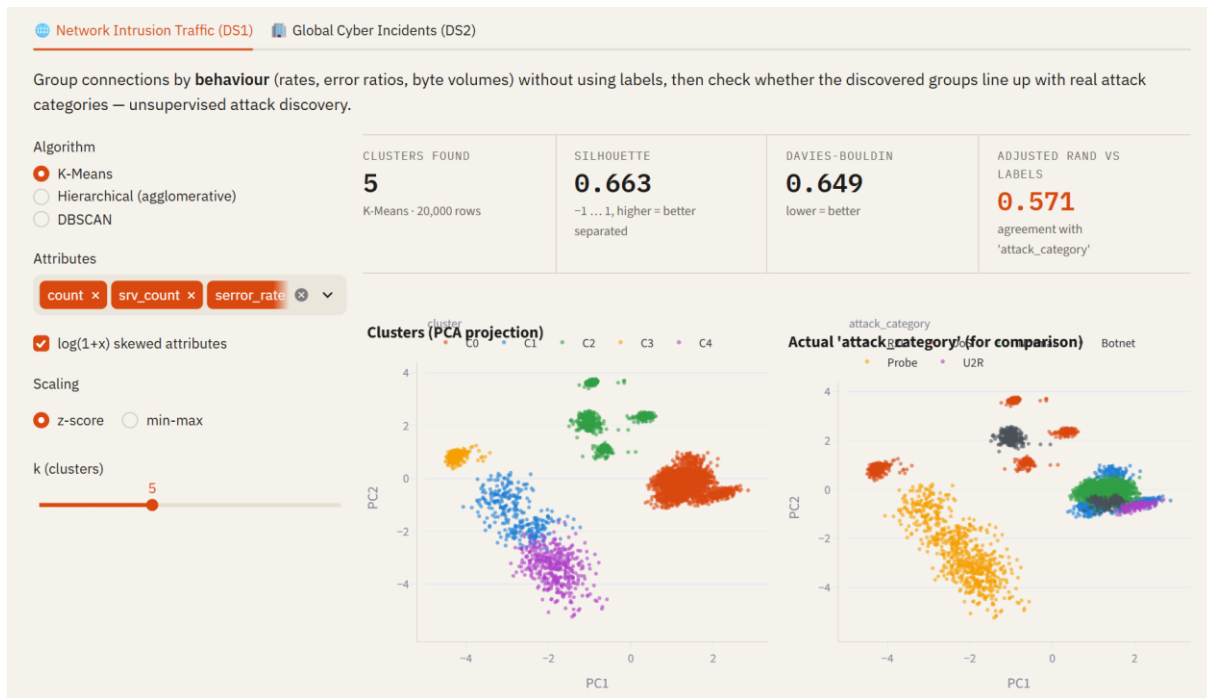


Figure 23. K-Means on DS1: cluster quality metrics and PCA views of clusters versus actual labels.



Figure 24. Elbow method and silhouette coefficient for $k = 2 \dots 10$.

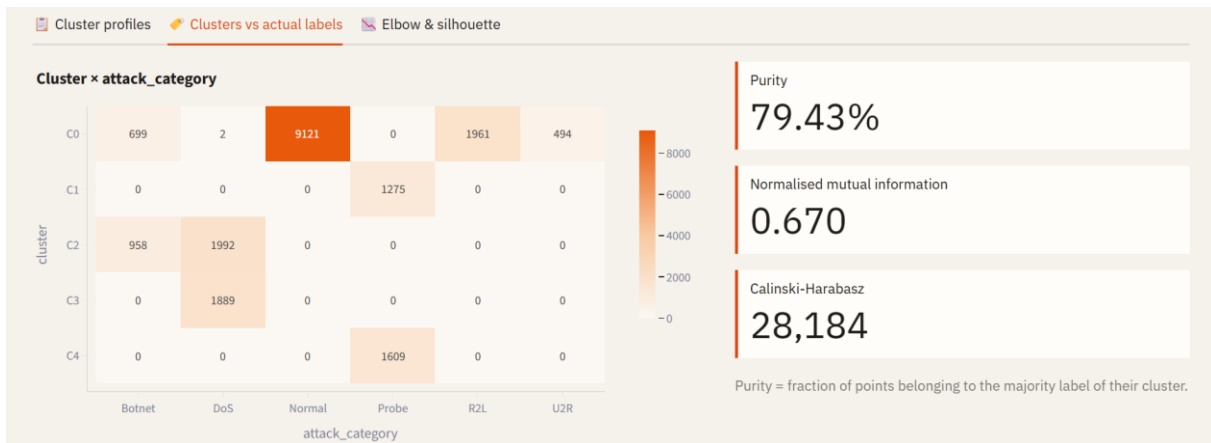


Figure 25. Cluster × attack_category cross-tab with purity, NMI and Calinski-Harabasz score.

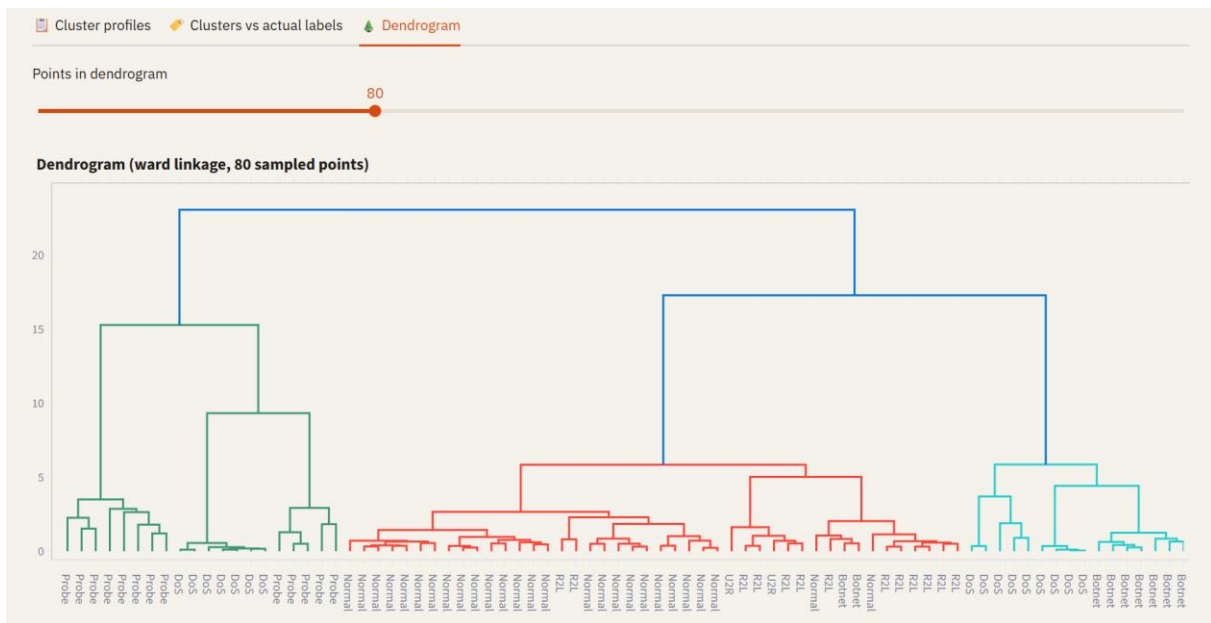


Figure 26. Dendrogram of 80 sampled connections (ward linkage).

8. Comparison, Recommendation and Deployment

Comparison & Best Model

The Model Comparison page trains J48 and Naive Bayes on the same data and the same test protocol as a ZeroR baseline and three other classifiers (k-NN with $k = 5$, logistic regression and random forest). It then picks the model with the highest weighted F1. Association rules, classification, regression and clustering solve different problems, so they are not ranked on one common score; each is judged within its own task.

Table 13. Classifier comparison on DS1 (10,000 sampled rows, 66% split).

Model	Accuracy	Kappa	Precision	Recall	F1	ROC area
ZeroR (baseline)	45.56%	0.000	0.208	0.456	0.285	0.500
J48 (C4.5)	99.00%	0.986	0.990	0.990	0.990	0.994
Naive Bayes	87.21%	0.819	0.875	0.872	0.861	0.987
k-NN (IBk, $k = 5$)	99.32%	0.991	0.993	0.993	0.993	0.999
Logistic regression	95.47%	0.936	0.954	0.955	0.954	0.996
Random forest	99.59%	0.994	0.996	0.996	0.996	0.9995

Table 14. Classifier comparison on DS2 (5,000 incidents, 66% split).

Model	Accuracy	Kappa	Precision	Recall	F1	ROC area
ZeroR (baseline)	42.35%	0.000	0.179	0.424	0.252	0.500
J48 (C4.5)	76.76%	0.655	0.769	0.768	0.768	0.851
Naive Bayes	74.35%	0.603	0.766	0.744	0.737	0.894
k-NN (IBk, $k = 5$)	65.12%	0.468	0.646	0.651	0.643	0.802
Logistic regression	81.41%	0.719	0.818	0.814	0.814	0.940
Random forest	82.88%	0.740	0.835	0.829	0.826	0.940

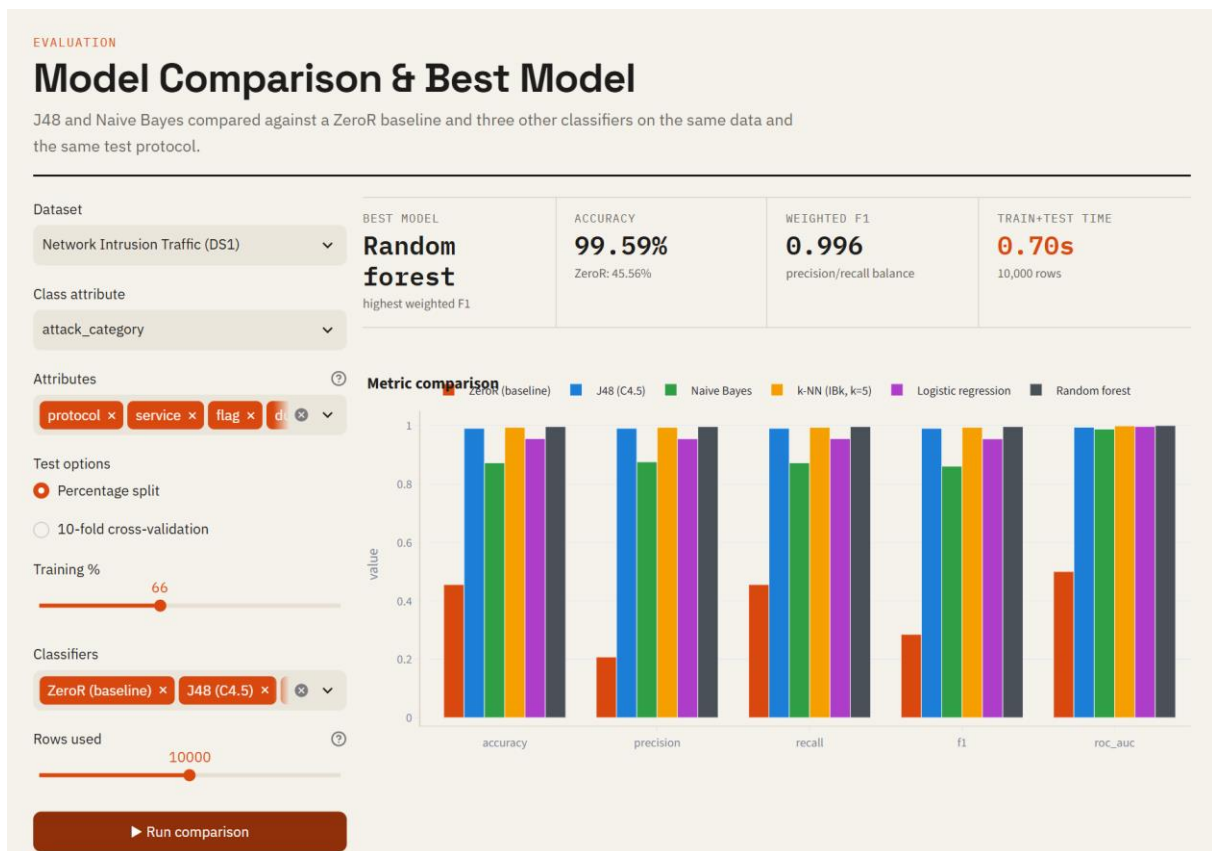


Figure 27. Model Comparison page: best model, KPIs and the metric comparison chart (DS1).

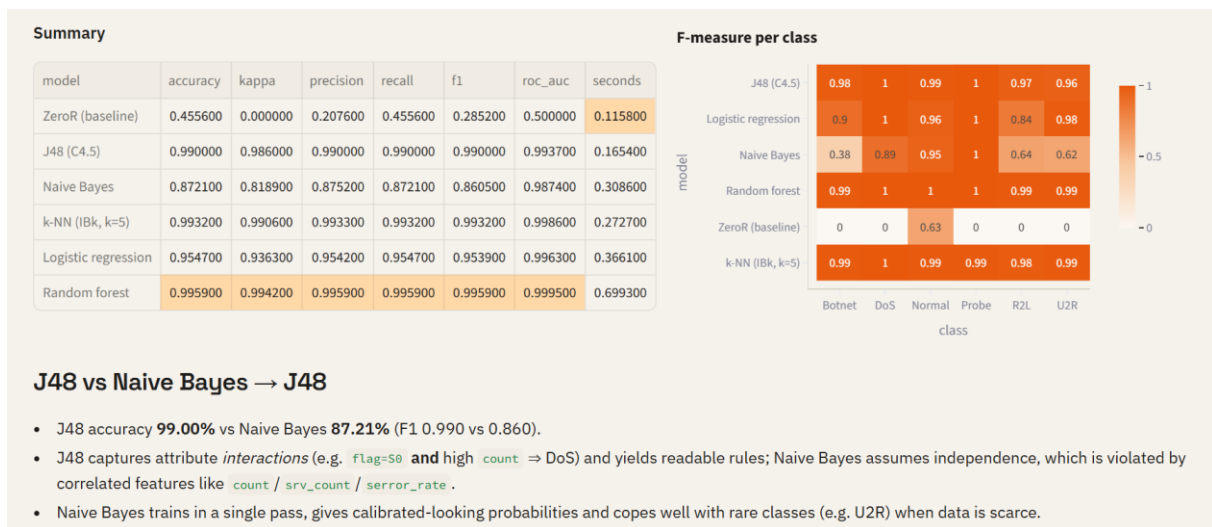


Figure 28. Summary table, F-measure per class and the J48 versus Naive Bayes verdict.

Task-specific Recommendation

Table 15. Best-suited technique for each task.

Task	Best-suited model	Reason (measured)
Classification (required pair)	J48	Beats Naive Bayes on both datasets (DS1 F1 0.995 vs 0.862; DS2 0.768 vs 0.737) and gives readable IF-THEN rules.
Classification (all models)	Random forest	Highest F1 on both datasets (0.996 on DS1, 0.826 on DS2), at the cost of interpretability and run time.
Probability ranking on DS2	Naive Bayes / logistic regression	Higher ROC area than J48 (0.894 / 0.940 vs 0.851); useful when incidents are ranked for triage.
Regression	Multiple linear regression	R^2 0.886 (DS1) and 0.747 (DS2) with an interpretable equation; simple regression on one predictor is much weaker.
Clustering	K-Means, k = 5 (DS1)	Best silhouette (0.663) with 79% purity; hierarchical ward is close (ARI 0.595) but does not scale to all rows.
Association rules	Apriori or FP-Growth	Identical itemsets; at 51 items Apriori was faster, but FP-Growth scales better at low support.

9. Testing, Limitations and Future Scope

Testing Checklist

The application was run locally on 9 October 2026, and every page was opened and checked against the outputs reported above. Items marked 'To verify' were not exercised in that run.

Table 16. Testing checklist.

#	Test	Result
1	Default datasets load and their dimensions are shown ($30,180 \times 23$ and $5,000 \times 16$).	Passed
2	Missing values and duplicates are detected and handled ($1,050 \rightarrow 0$ missing; $30,180 \rightarrow 30,000$ rows).	Passed
3	Star schema, roll-up and dice views render with correct row counts.	Passed
4	Association rules run with both algorithms; support, confidence and lift are displayed.	Passed
5	J48 and Naive Bayes produce metrics, confusion matrices and per-class tables.	Passed
6	Regression runs on DS1 and DS2 with R^2 , adjusted R^2 , RMSE and MAE.	Passed
7	K-Means, hierarchical clustering and the elbow / dendrogram views run.	Passed
8	Model comparison uses actual results and names the best model.	Passed
9	Upload of a separate CSV / Excel file and its validation.	To verify
10	Deployed application opens and its main pages work.	To verify

Limitations

- Both datasets are synthetic. They follow realistic schemas, but their patterns may be cleaner than real traffic, which partly explains the near-perfect DS1 accuracy.
- The J48 page uses a scikit-learn decision tree configured like J48, not Weka's own implementation, so results can differ slightly from Weka (the ARFF export allows a direct check).
- Association rules show co-occurrence, not causation, and depend on the chosen discretisation bins.
- K-Means needs k in advance and is sensitive to scaling and outliers; hierarchical clustering and DBSCAN are run on samples (4,000 and 8,000 rows) to stay within cloud memory limits.
- Streamlit Community Cloud has limited CPU and memory, so the comparison page samples rows to keep training quick.

Future Scope

- Evaluate the system on real public intrusion datasets such as NSL-KDD, CIC-IDS2017 or UNSW-NB15.
- Add more algorithms (gradient boosting, SVM, isolation forest for anomaly detection) and hyper-parameter tuning.
- Stream live logs into the warehouse for near-real-time detection and alerting.
- Add explainability (SHAP values) and export of the analysis results as a report.
- Add authentication and access control if the application is used beyond demonstration.

Conclusion

The project shows how the data warehousing and data mining techniques of the course fit together on one problem. Preprocessing and the star-schema warehouse turn noisy connection logs into an analysable cube, and association rules expose interpretable attack signatures that differ by network. On DS1, the J48-equivalent decision tree classifies attacks with 99.47% accuracy against 87.41% for Naive Bayes. Regression predicts traffic volume (R^2 0.886) and incident loss (R^2 0.747), and unsupervised K-Means recovers most attack categories with 79% purity. Each task has its own best-suited model, so the comparison page recommends a model per task instead of declaring one algorithm universally best.

References

- 1.J. Han, M. Kamber and J. Pei, Data Mining: Concepts and Techniques, 3rd ed., Morgan Kaufmann.
- 2.J. R. Quinlan, C4.5: Programs for Machine Learning, Morgan Kaufmann, 1993.
- 3.M. Tavallaee et al., "A detailed analysis of the KDD CUP 99 data set", IEEE CISDA, 2009 (NSL-KDD).